第27天 · 用缓存与并发控制重复工作

通过实际调用数观察优化效果，同时保留预算和结果正确性。

标准路线：60分钟；忙碌日：20分钟；扩展练习另计。

今天的目标

- 比较重复查询在缓存开关前后的实际调用数。

- 解释并发数与调用总预算的不同含义。

- 指出缓存正确性与多用户隔离的后续要求。

开始之前

- 已经完成第 26 课，或能解释其产物。

- 打开实践包 practice 目录；后文 python 可替换为虚拟环境内解释器的完整路径。

一小时安排

- 15分钟 · 理解概念：阅读本课四节讲解，弄清一个关键机制与适用边界。

- 30分钟 · 亲手操作：先跑基线，再做一次单变量实验，保留前后结果。

- 10分钟 · 检查结果：逐项验收，完成两道自测并阅读解析。

- 5分钟 · 留下记录：写出本课发现、未解决问题和下次第一步。

01 · 理解概念

缓存保存的是特定条件下的结果

同一个查询重复出现时，可以复用已计算的检索结果，减少重复工作。缓存键应能代表影响结果的条件，例如查询、检索参数和材料版本。

本课的缓存是一次教学批处理中的本地机制。它不是跨机器长期缓存，也没有实现多用户敏感数据的完整隔离。未来扩展时，材料更新、用户身份和过期策略都可能成为缓存键或失效规则的一部分。

并发适合相互独立的工作

多个互不依赖的查询可以同时检索，max_workers限制并发工作者数量。若后一步依赖前一步结果，就不能只为了加速而把它们并行执行。

并发会增加共享状态的复杂度，例如两个相同查询同时到达时，是否重复调用或争用缓存。应通过actual_calls与cache_hits观察当前实现的行为，而不凭代码里出现线程池就断言更快。

调用预算约束资源消耗

max_calls限制实际计算或调用次数。缓存命中通常可减少实际调用，但预算耗尽后必须返回明确状态，不能偷偷突破上限来凑齐结果。

本练习是小型词法检索，单次工作很轻，线程开销可能抵消并发收益。课程重点是理解资源控制与可观察行为，不保证某台电脑上的耗时一定下降。

优化之后仍要检查结果

优化应保留查询与结果的对应关系，不能因为并发完成顺序不同就把答案配错问题。对重复查询还要检查引用和证据ID的一致性。

现实应用中，缓存可能保存过时资料；不同用户的相同问题也可能有不同权限。将这些限制写进扩展计划，再决定是否需要持久缓存、租户隔离、过期与失效机制。

02 · 一步一步做

命令在解压后的 practice 文件夹执行。Windows可使用 .\.venv\Scripts\python.exe；macOS/Linux可使用 .venv/bin/python 替代 python。

1. 找到本课的输入与配置

为什么做：先知道程序读什么，后面才能解释输出为何改变。

在 VS Code 中打开 experiments.json，定位 day27。用下面的命令确认当前目录正确，并把各参数的作用写在笔记中。

```
python -c "import json; print(json.load(open('experiments.json', encoding='utf-8'))['day27'])"
```

预期结果：终端显示本课配置；键名与文件中的内容一致。

2. 运行本课基线

为什么做：保持默认设置的一次运行，是后续比较的参照。

执行命令，打开新生成的 outputs/day27.json。先检查 day、mode 和 result，不急着修改代码。

```
python run.py day 27
```

预期结果：结果写入 outputs/day27.json；主线 mode 明确为 offline，代表规则教学流程。

3. 沿输出检查机制

为什么做：不能只看程序有没有报错，要检查它实际做了什么。

格式化阅读输出，重点检查results、actual_calls、cache_hits 与 budget_exhausted。把一个输入参数和一个输出字段用箭头对应起来。

```
python -m json.tool outputs/day27.json
```

预期结果：能够定位results、actual_calls、cache_hits 与 budget_exhausted，并用自己的话解释该结果。

4. 改变一个条件并比较

为什么做：主动制造可解释的变化，才能验证自己理解的机制。

完成下面“改一改”实验。对比基线与新结果，记录哪些字段改变、哪些保持稳定；不要同时改模型、材料和参数。

预期结果：留下一组前后结果和一句解释；失败状态也可以是正确的实验产物。

改一改 · 关闭缓存，观察重复查询是否重复执行

- 先执行 python run.py day 27，将 outputs/day27.json 复制为自己的基线备份。

- 在 VS Code 打开 experiments.json，只修改 day27.cache 为 false，保持其他键不变并保存。

- 执行 python -m json.tool experiments.json 检查语法，再执行 python run.py day 27。

- 对比两份结果，重点观察results、actual_calls、cache_hits 与 budget_exhausted；解释改变原因后恢复原设置或明确记录保留值。

观察什么：保持默认的三个查询和max_calls=3不变；关闭缓存后重复问题不再命中缓存，实际调用次数应相应增加，结果仍应对应原查询。

机制解释：单变量比较把重复计算带来的差异与并发数量、调用预算分开，避免混淆优化原因。

常见问题

修改配置后结果没有变化

原因：改了其他天的字段，或编辑后尚未保存。

处理：核对 experiments.json 中的 day27，保存后重新运行；检查实际执行目录。

把受控失败误当成程序坏了

原因：本课实验主动制造了边界条件。

处理：先对照 expected 和状态字段，区分设计内拒绝与未处理异常；保留错误信息再修复。

03 · 验收

通过实际调用数观察优化效果，同时保留预算和结果正确性。 保留基线、修改结果与自己的解释。

成果文件：outputs/day27.json

☐ 能从查询列表找出重复输入。

☐ 保存缓存开启与关闭的调用数及命中数。

☐ 检查结果对应关系，并写出一个缓存失效或隔离风险。

自测与解析

1. max_workers=2与max_calls=3分别约束什么？

- A. 都表示总任务数

- B. 前者是并发数量，后者是实际调用预算

- C. 前者是缓存天数，后者是模型参数

- D. 都表示费用

答案：B。 同时做多少与累计做多少是两项不同控制。

2. 缓存命中率更高是否总能说明结果更好？

- A. 总是

- B. 不一定，还要检查过期、权限与正确性

- C. 只要快就好

- D. 命中就不需要来源

答案：B。 缓存可能返回过时或权限不合适的结果，效率指标不能替代正确性检查。

小词典

- Cache hit：请求复用了已有缓存结果。

- Concurrency：多个任务在时间上重叠执行。

- Call budget：一次任务允许产生的实际调用上限。

04 · 我的记录

- 我的任务里哪些步骤独立，哪些必须等待上一步？

- 资料更新以后，我怎样避免继续使用旧缓存？

忙碌日：20分钟

- 用5分钟阅读第一个概念，写出本课要解决的问题。

- 用10分钟运行 python run.py day 27，打开 outputs/day27.json，检查一个关键字段。

- 用5分钟完成第一道自测并留一句解释，完整实验与验收留到下次。

保存一份运行结果和自己的解释；精简练习单独记录，不等于整课验收全部完成。

有余力再做：20分钟选学 · 把本课机制迁移到一个自己的任务

- 从自己的文献整理或数据分析流程中选一个相似环节。

- 写清输入、允许动作、失败出口与验收方法。

- 提出一个会让当前实现失败的反例，并标明它是待实现扩展。

参考资料与阅读范围

Python：concurrent.futures：阅读ThreadPoolExecutor、submit与结果收集，观察独立工作如何并发。
https://docs.python.org/3/library/concurrent.futures.html

BioAgentLab：科研 Agent 教材：结合状态与记忆章节，思考数据生命周期和隔离。
https://github.com/zbyloveyj/BioAgentLab

下一步

下一课为源码、数据与依赖建立可复现的交付清单。